Building Agentic Workflows with AmSC MAG and IRI

Genesis Mission October Hackathon

Huihuo Zheng
AmSC AI Services
Argonne Leadership Computing Facility
Argonne National Laboratory

October 14, 2026

huihuo.zheng@anl.gov

github.com/zhenghh04/gm_hackathon_agentic_ai

Scientists spend their first hours on mechanics, not on science

The manual loop today

1. SSH to a login node
2. Read module docs, hand-write a script
3. Fix account, queue, walltime, paths
4. Submit, then poll `qstat`
5. Parse the error, edit, resubmit
6. Move the output files by hand

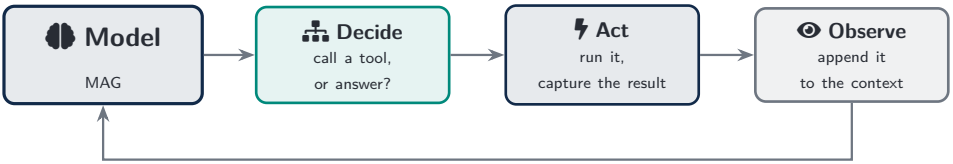
The same run, delegated

"Run my training script on two Polaris nodes and tell me when it converges."

- Resolves account, queue, paths
- Stages the script and submits
- Monitors; diagnoses failures
- Reports back with evidence

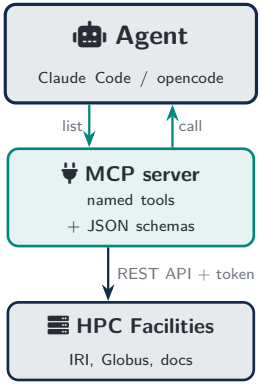
The mechanics are the bottleneck — and every one of them is already an API. This tutorial wires an agent to those APIs.

An agent is a loop: a model, some tools, a stopping condition



A chatbot only *answers*; this loop lets the model **act** — call a tool, observe the result, decide again. That is the shift from chatbot to agent.

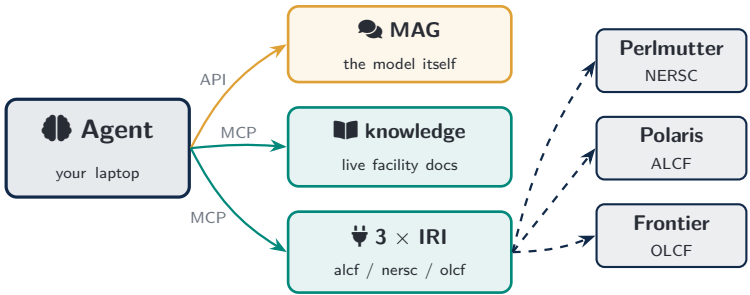
Model Context Protocol (MCP): how the agent reaches a facility through APIs



What MCP buys you

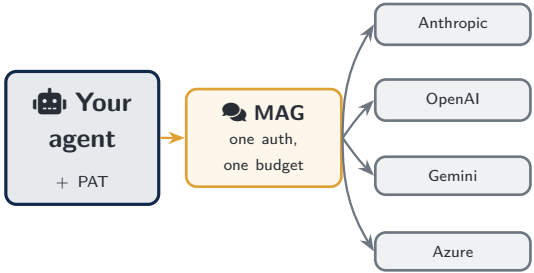
- **Discovered.** The server wraps the facility's REST API as named tools; the agent asks what it can do — no URLs, no hand-built payloads.
- **Typed.** One schema per tool, so bad arguments fail *before* a job runs.
- **Scoped by you.** The server holds the token and decides what is reachable.
- **Configured once.** `.mcp.json` for Claude Code, `opencode.jsonc` for opencode.

Building blocks: Agent core (MAG), knowledge base, IRI MCP



Knowledge, reach, data — and judgment. The fourth leg is the `submit-job` skill.

MAG: Model Access Gateway to frontier models



The **Model Access Gateway** is one public-cloud endpoint that fronts every model AmSC exposes behind **one auth layer, one budget, and one catalog**. Your **PAT** replaces every provider API key — usage is billed to your project, with no VPN, no personal key, and no credit card.

Where to get your PAT

(Personal Access Token)

1. Log in to `https://my.amsc.energy.gov`
2. Under **Project**, select *Genesis Mission October 2026 Hackathon*
3. **Personal Access Tokens** → **New Token**
4. Paste it into `~/.claude/settings.json` — next slide.

Set it once: `~/.claude/settings.json`

paste your token where `YOUR_PAT_HERE` is — then every shell inherits it

```
{
  "env": {
    "ANTHROPIC_BASE_URL": "https://mag.genesis.americansciencecloud.org",
    "ANTHROPIC_AUTH_TOKEN": "YOUR_PAT_HERE",
    "ANTHROPIC_CUSTOM_HEADERS": "X-AmSC-MAG-Provider: amazon"
  },
  "model": "opus"
}
```

depending on where you sit, MAG may need your lab VPN or an HTTP(S) proxy — check with `curl -i https://mag.genesis.americansciencecloud.org/v1/models`

A **401** means you reached the gateway and it just wants your PAT — that is the *good* answer. A hang or TLS error is the network, not the token: connect the VPN, or set `HTTPS_PROXY`.

...and for Codex: ~/.codex/config.toml

Codex speaks the OpenAI wire — point it at MAG's `/v1` with a custom provider

```
model = "gpt-5@openai"
model_provider = "amsc"

[model_providers.amsc]
name = "AmSC MAG"
base_url = "https://mag.genesis.americansciencecloud.org/v1"
env_key = "AMSC_TOKEN"
wire_api = "responses"
```

same PAT, same gateway — only the wire differs; `@openai` pins the provider *and* the bucket it bills

```
export AMSC_TOKEN=YOUR_PAT_HERE
codex
```

Codex 0.155+ *removed* the `chat` wire — `wire_api = "chat"` now refuses to load. MAG serves `responses` natively, so this config is the current one.

The knowledge leg: ask the facility's own docs

mcp/knowledge/docs_server.py — a server that is a client

```
ASK_ALCF = "https://ask.alcf.anl.gov/mcp"

@mcp.tool()
async def retrieve_alcf_docs(query, top_k=3):
    # we are a CLIENT of ALCF's public server
    async with Client(ASK_ALCF) as up:
        return await up.call_tool(
            "retrieve_alcf_docs",
            {"query": query, "top_k": top_k})
```

Already the `knowledge` server in `.mcp.json` — the URL is the entire configuration.

What it knows

AskALCF

- ALCF, OLCF, NERSC & LLNL docs — plus PBS, Slurm, CUDA, HIP, oneAPI, SYCL, OpenMP.
- **No token, no account.** It answers before you have logged in to any facility.
- A server that is a *client* — one `.mcp.json` entry, and it routes around the Cloudflare edge (which 403s Node `fetch` but lets Python's HTTP stack through).

Ask a facility question and watch for the `retrieve_alcf_docs` call — no call means training data, not live docs.

IRI for compute: the facility REST API, wrapped as named tools

mcp/hpc/alcf_server.py — one tool

```python
@mcp.tool()
async def submit_job(
    resource_id: str, # which machine
    executable: str, # /bin/bash
    arguments: str = "",
    queue_name: str = "debug",
    account: str = "",
    duration: int = 300,
    node_count: int = 1) -> str:
    """Submit a batch job via IRI.
    duration is wall-clock SECONDS.
    account is billed - never guess.
    """
```

The docstring *is* the tool description the model reads.

Six IRI tools

- `get_system_status` — is the machine up?
- `submit_job` — a batch job via IRI
- `get_job_status` — poll one job
- `list_jobs` — filter by state / queue
- `cancel_job` — stop a run, and the billing
- `read_file` — fetch stdout / stderr

The same names on `alcf-iri` (PBS), `nersc-iri` and `olcf-iri` (Slurm) — the agent picks the server from the system you name, and each holds its own token the model never sees.

Capture the chain: the `submit-job` skill

Generate, stage, submit, monitor, report — written down once.

.claude/skills/submit-job/SKILL.md

```
---
name: submit-job
description: Stage inputs, run an IRI batch
  job, monitor it, and report back.
---
1. Generate run.sh locally; show it first.
2. globus_transfer, then poll until
   SUCCEEDED. A task ID is not a delivery.
3. get_system_status, then submit_job.
4. Poll with backoff: 10s, then 30s.
5. Quote the stderr line; propose a fix.
   Never resubmit on your own.
```

what is a skill?

- **A repeatable workflow, written down** — the steps you would otherwise retype every session.
- **Just a markdown file** — it diffs and ships like any other artifact.
- **The `description` routes it** — that line is how the agent decides to load it.

One file spans the whole chain — the skill is where it lives, not the prompt.

The key source files: four servers and one skill

what you cloned

```
gm_hackathon_agentic_ai/
 .mcp.json              4 servers, one file
 mcp/
  auth_env.py           tokens, Globus id
  knowledge/
   docs_server.py        1 tool  AskALCF
  hpc/
   alcf_server.py       23 tools Polaris
   nersc_server.py      20 tools Perlmutter
   olcf_iri_server.py   17 tools Frontier
   *_iri_client.py      REST + Globus calls
 .claude/skills/
  submit-job/SKILL.md   the chain
 scripts/auth/          one login per site
```

The shape to copy

- **Server declares, client calls.** `*_server.py` is tool names and schemas; `*_iri_client.py` does the HTTP and Globus work.
- **Auth lives in one file.** `auth_env.py` reads `.env`, and resolves your laptop's Globus collection for `local_endpoint`.
- **Three facilities, one shape.** Same tool names on `alcf` / `nersc` / `olcf` — learn one, use all three.

Two folders, the whole system — `mcp/` for reach, `.claude/skills/` for judgment.

Hands-on

Step 1: clone, configure, and prove the model answers

macOS / Linux / WSL

```
git clone https://github.com/zhenghh04/\
gm_hackathon_agentic_ai.git
cd gm_hackathon_agentic_ai
python3 -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt
read -rs AMSC_TOKEN # codex/opencode only
export AMSC_TOKEN # Claude: see settings.json
claude # ...or: opencode
```

native Windows (PowerShell) — only these lines differ

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
$env:AMSC_TOKEN = Read-Host 'PAT'
```

~/.claude/settings.json

```
{
  "env": {
    "ANTHROPIC_BASE_URL": "https://mag.genesis.
          americansciencecloud.org",
    "ANTHROPIC_AUTH_TOKEN": "YOUR_PAT_HERE",
    "ANTHROPIC_CUSTOM_HEADERS": "X-AmSC-MAG-
          Provider: amazon"
  },
  "model": "opus"
}
```

Claude Code reads the PAT from this file — no `export` needed. The shell variable on the left is for codex and opencode, which read it from the environment.

Set once, every shell inherits it — then Lab 01's round-trip check confirms the model answers.

Native Windows: `%USERPROFILE%\.claude\settings.json`. Under WSL it is `~/.claude/` *inside* the distro — pick one and stay there.

Add `-MaskInput` to hide the PAT — PowerShell 7.1+ only.

Globus Connect Personal: your laptop as a transfer endpoint

Download the package for **your OS and architecture**, run the guided setup — it creates your personal collection.

downloads.globus.org/globus-connect-personal/

Which build? — the usual confusion

- `x86_64` **and** `amd64` name the *same* arch — the only Linux build. `uname -m` prints `x86_64`.
- **Apple Silicon Mac:** install the universal `.dmg`, *not* the Linux tarball. A `uname -m = arm64` warning means you grabbed the Linux build on a Mac.

Before you share

The default collection exposes your *whole home directory*, read-write. Narrow it in `~/.globusonline/lta/config-paths`.

Docs: `globus.org/globus-connect-personal`

Step 2: one login per facility mints the IRI tokens

one command per facility — Globus Transfer + the IRI token, together

```
python scripts/auth/globus_auth.py authenticate --facility alcf # -> IRI_TOKEN_ALCF
python scripts/auth/globus_auth.py authenticate --facility nersc # -> IRI_TOKEN_NERSC
```

Two labelled logins each; NERSC also asks you to link a `nersc.gov` identity.

OLCF has no browser flow — paste a portal token

```
# my.olcf.ornl.gov -> Projects -> API Tokens (enable the COMPUTE scope!)
echo "IRI_TOKEN_OLCF=<paste-the-token>" >> .env
```

Where the tokens go

Into `.env`, re-read on every MCP call — never into the model's context. Rotate one and the next call picks it up.

then prove it

`/mcp` in the agent lists all four servers (on opencode: `opencode mcp list`). Only do the facilities you have — a missing token just reports "not authenticated".

Step 3: first contact — ask the system, not the model

in the session

```
> What model are you?
/model # what is ACTUALLY configured
/mcp # the four servers and their tools

> How many nodes does Polaris have?
  Which queues can I use?
```

Codex: same — `/model`, `/mcp`.

opencode: `/models`; no `/mcp` slash command — run `opencode mcp list` in a shell.

what to notice

- **Its answer is a guess.** Ask the system — `/model` — not the model.
- **`/mcp` is the real check.** Not listed there, nothing later works.
- **The last one calls a tool.** The `knowledge` server answers from live docs — no tool call means you got training data.

Example 1: read-only questions cost no node-hours

You

"What is the status of Polaris and Perlmutter right now, and what were my last five jobs to finish?"

</> the calls it makes — watch which, and in what order

```
get_system_status(system="polaris") # up/down, current reservations
get_system_status(system="perlmutter") # a DIFFERENT server, same shape
list_jobs(system="polaris")
```

Start here, every time. Read-only tools are how you find out whether the plumbing works before a mistake costs an allocation.

Example 2: a two-node smoke test, submitted and watched

You

"Submit a two-node hello-world to the Polaris debug queue, then tell me what it printed."

</> what the skill makes it do

```
list_projects() # resolve the account
submit_job(system="polaris", nodes=2,
          queue="debug",
          walltime="00:10:00", ...)
get_job_status(job_id=...) # poll
read_file(path=".../job.out", tail=40)
```

This one spends real time

- The account came from `list_projects`, not from the model's memory.
- Read the *tail* of the log, not the whole file — context is the scarce resource.

Example 3: one sentence → a MNIST job on a Polaris GPU

You

"Please run a MNIST training on Polaris using <ALLOCATION> on 1 node, debug queue."

</> write → stage → submit → monitor

```
# from one sentence, the agent writes code:
Write("train.py") # PyTorch MNIST, A100-ready
Write("run.sh") # #!/bin/bash -l, module load
globus_mkdir(dst=".../mnist/") # make folder
globus_transfer(src="train.py", dst=
  "/eagle/<project>/<you>/mnist/")
globus_transfer_status(task_id=...) # SUCCEEDED
submit_job(system="polaris", nodes=1,
  queue="debug", walltime="00:15:00",
  account="<ALLOCATION>")
get_job_status(job_id=...) # poll, backoff
read_file(path=".../job.out", tail=20)
```

What the job printed

```
Epoch 1/5 loss 0.42 acc 95.1%
Epoch 2/5 loss 0.15 acc 97.3%
Epoch 3/5 loss 0.10 acc 98.0%
Epoch 4/5 loss 0.07 acc 98.4%
Epoch 5/5 loss 0.05 acc 98.6%
Final test accuracy: 98.6%
```

Illustrative `job.out` tail — numbers vary run to run.

Take it home: now run your own workload

What you just did

- Pointed one agent at every model through MAG.
- Reached three facilities by name through four MCP servers.
- Staged, submitted, and monitored a real HPC job — from one sentence.

Now do it with your own

Swap the MNIST example for *your* code, *your* allocation, *your* queue — nothing here is tutorial-specific. Point the same servers at your project and run the workload you actually came with.

github.com/zhenghh04/gm_hackathon_agentic_ai · huihuo.zheng@anl.gov

Acknowledgement

This research used resources of the Argonne Leadership Computing Facility, a U.S. Department of Energy (DOE) Office of Science user facility at Argonne National Laboratory and is based on research supported by the U.S. DOE Office of Science-Advanced Scientific Computing Research Program, under Contract No. DE-AC02-06CH11357.

Backup

Backup: no MAG? Point Claude at ALCF Inference Endpoints

instead of the MAG PAT in Step 1

```
# clone / venv / pip --- same as Step 1, then:
alcf-tokens login # browser: "Argonne LCF"
alcf-tokens test-token inference
# {"ready": true, "error": null}
uvx alcf-ai agent configure claude
# ...or: configure opencode
claude
```

What changes — and what doesn't

- `alcf-tokens login` mints your ALCF *inference* token; `test-token inference` is the only proof it works.
- `alcf-ai agent configure claude` writes `~/.claude/settings.json` *for* you — aimed at the ALCF Inference Service. No PAT, no hand-edited JSON.
- **Everything else is identical** — the four MCP servers, the `submit-job` skill, Examples 1–3. ALCF inference speaks the same Anthropic wire as MAG.

Needs an ALCF account. Guide:

`docs.alcf.anl.gov/services/inference-endpoints`

Backup: prove the endpoint answers — no agent needed

>_ a one-line round trip to the ALCF Inference Service

```
token=$(alcf-tokens get-token inference)
curl -H "Authorization: Bearer $token" \
    -H "Content-Type: application/json" \
    -d '{"model": "gpt-oss-120b",
         "messages": [{"role": "user", "content": "hello!"}]}' \
  https://inference-api.alcf.anl.gov/resource_server/metis/api/v1/chat/completions | jq
```

Same token the agent uses — if this returns a reply, the endpoint is healthy and the agent will reach it too.

`alcf-ai ls-jobs metis` shows which models are hot right now.

References

Reference: ALCF *Service Enabled Science* training

`github.com/argonne-lcf/Service_Enabled_Science`

Session 05, *Agentic Workflows* — slides:
`.../blob/main/05_Agentic_Workflows/slides.pdf`

This tutorial's MCP-server layout, the `submit-job` skill, and the ALCF-inference backup all build on that session.